

Problem 1

Design a class named **BankAccount** to simulate a simple banking system.

Each account should contain the following information:

- **Account name** (default: "anonymous")
- **Account balance** (default: 0.0)
- **Account type** ("saving" or "current", default: "saving")

The class should support the following operations:

1. **deposit(amount)** - Add a given amount to the account balance.
2. **withdraw(amount)** - Withdraw a given amount from the account balance; if the balance is insufficient, display a warning message.
3. **display()** - Print or return all account details (name, balance, and type).
4. **Equality test (==)** - Return True if two accounts belong to the same account name.
5. **Inequality test (!=)** - Return True if two accounts belong to different account names.

There may be multiple "anonymous" accounts at the same time, but their balances and transactions must remain completely independent.

Problem 2

Implement a class named **Vector3D** to represent a 3-dimensional vector of floats.

The class should provide the following features:

1. **Constructors:**
 - o A default constructor that accepts three values from standard input.
 - o Another constructor that accepts an array of size 3.
2. **Member functions:**
 - o `add(const Vector3D& v)` - returns the vector sum.
 - o `dot(const Vector3D& v)` - returns the dot product.
 - o `cross(const Vector3D& v)` - returns the cross product.
 - o `print()` - prints the vector in the form (x, y, z).
3. **Operator Overloading:**
 - o `operator+` - vector addition.
 - o `operator-` - vector subtraction.
 - o `operator*` - scalar multiplication.
 - o `operator+=` - adds another vector to the current one.
 - o `operator[](int index)` - provides subscript access to x, y, or z.
 - Returns `float&` for non-const objects and `float` for const objects.

Each vector's components (x, y, z) should be independent of others.

Problem 3:

Create two classes, **Author** and **Publisher**, both without default constructors. Each class should have a constructor that takes a string argument and stores it in an internal string member.

Implement the **copy constructor** and **assignment operator** for each class properly.

Then, create a derived class called **Book**, which inherits from **Author** and contains a member object of type **Publisher**.

Write:

- a **default constructor**,
- a **constructor that takes two string arguments** (one for the author name and one for the publisher name),
- a **copy constructor**,
- and an **assignment operator** for the Book class.

Problem 4:

Write a base class that has:

- one **virtual function** named `process()`, and
- one **non-virtual function** named `compute()`.

Then derive a class called **AdvancedProcessor** that overrides the virtual function and redefines the non-virtual one (hides it).

Create an object of `AdvancedProcessor`, upcast it to a pointer of the base class, and measure the execution time of repeated calls to both functions (`process()` and `compute()`) using the chrono library (`<chrono>`).

Compare and print out the average time difference between virtual and non-virtual calls.

Problem 5:

Design a base class `Shape` with a virtual function `clone()` that creates and returns a copy of the current shape.

Derive two subclasses, `Circle` and `Rectangle`, that each override `clone()` to return copies of their own type.

In the `main()` function:

- Create a vector of `Shape*` objects containing a `Circle` and a `Rectangle`.
- Iterate through the vector and call `clone()` on each element.
- Verify that each cloned object preserves its original type using `typeid` or `dynamic_cast`.

Then, modify the `clone()` function to return a `unique_ptr<Shape>` instead of a raw pointer.

Discuss the advantages of using smart pointers in this context.

Problem 6: Generic Minimum Finder with Custom Comparison

Description:

Design a function template `findMin` that finds the *smallest* of three given values of various types.

If multiple elements share the same minimum value, the function should return the **last** one that attains it.

You need to support the following data types:

- int
- double
- char
- string (shorter string is smaller)
- Point structure (with x and y as integers, smaller *Euclidean distance* is smaller)
- Point* (compare by dereferencing pointer values)

Example input:

```
1 2 3
1.3 2.4 0.6
a b c
hello hi he
1,2 3,4 0,1
1,2 0,1 3,4
```

Expected output:

```
1
0.6
a
he
0,1
0,1
```

Hint:

- Define struct Point { int x, y; double dist() const; };
- Implement operator < or overload template for custom comparison.
-